



Python Programming - 2301CS404

Lab - 12

Roll No : 315

Name : Vishva M. Bhimani

24010101026

Exception Handling

01) WAP to handle following exceptions:

1. ZeroDivisionError
2. ValueError
3. TypeError

Note: handle them using separate except blocks and also using single except block too.

```
In [7]: a=2
b=0
try:
    c = a//b
    print(c)
except Exception as err:
    print(err)
print("end of the programme")
```

integer division or modulo by zero
end of the programme

```
In [10]: try:
a = int(input("enter any number :"))
```

```

    print(a)
except Exception as err:
    print(err)
print("end of the code")

```

invalid literal for int() with base 10: 'abc'
end of the code

In [11]: **try:**

```

    print("10"+10)
except Exception as err:
    print(err)
print("end of the code")

```

can only concatenate str (not "int") to str
end of the code

In [20]: **try :**

```

    a = int(input("enter any number for a"))
    b = int(input("enter any number for b"))
    c = a//b
    print(c)
    print(10+"10")
except Exception as err:
    print(err)
print("end of the code")

```

invalid literal for int() with base 10: 'abc'
end of the code

02) WAP to handle following exceptions:

1. IndexError
2. KeyError

In [19]:

```

d1 = [10,20,30,40,50,60]
try:
    index = int(input("enter index of the d1"))
    print(d1[index])
except IndexError :
    print("this is a index error")
except KeyError :
    print("this is a keyerror")
print("this is the end of the code")

```

this is a index error
this is the end of the code

03) WAP to handle following exceptions:

1. FileNotFoundError
2. ModuleNotFoundError

```
In [29]: try:
    fp = open("abc111.txt",'r')
    fp.read()
    print("file is readed")
    fp.close()

except FileNotFoundError as err:
    print("this is a filenot found error")
except ModuleNotFoundError as err:
    print("this module not found")
print("this is end of the code")
```

this is a filenot found error
this is end of the code

04) WAP that handles all type of exceptions in a single except block in standard error message format.

```
In [30]: try:
    a = int(input("enter any number for a"))
    b = int(input("enter any number for b"))
    c = a//b
    print(c)
except Exception as err:
    print(err)
print("end of the programme")
```

invalid literal for int() with base 10: 'abc'
end of the programme

05) WAP to demonstrate else and finally block.

```
In [34]: try:
    a = int(input("enter any number for a"))
    b = int(input("enter any number for b"))
    c = a//b
except Exception as err:
    print(err)
else:
    print(c)
finally:
    print("this finally block is always excuted")
print("end of the code")
```

invalid literal for int() with base 10: 'zcxc'
this finally block is always excuted
end of the code

06) WAP to create an udf divide(a,b) that handles ZeroDivisionError.

```
In [35]: def divide(a,b):
          c = a//b
          print(c)

          try:
              divide(1,0)
          except Exception as err:
              print(err)
          finally:
              print("this block is always excuted")
          print("this is a end of the code")
```

integer division or modulo by zero
 this block is always excuted
 this is a end of the code

07) WAP to accept item prices and calculate total. Raise ValueError if price is negative, otherwise print the total bill amount.

```
In [40]: try:
          n = int(input("Enter number of items: "))
          total = 0

          for i in range(n):
              price = float(input(f"Enter price of item {i+1}: "))

              if price < 0:
                  raise ValueError("Price cannot be negative.")

              total += price

          print("Total Bill Amount =", total)

          except ValueError as e:
              print("Error:", e)
```

Error: Price cannot be negative.

08) Create a short program that prompts the user for a list of grades separated by commas.

Split the string into individual grades and use a list comprehension to convert each string to an integer.

You should use a try statement to inform the user when the values they entered cannot be converted.

```
In [50]: grades = input("Enter grades separated by commas: ")

          try:
              grades = [int(grade.strip()) for grade in grades.split(",")]
```

```
print("Converted Grades List:", grades)

except ValueError:
    print("Error: Please enter only numbers separated by commas,the values entered
```

Error: Please enter only numbers separated by commas,the values entered cannot be converted.

09) Accept 5 subject marks (0-100). Use assert for validation.

Sample Input (Error Case)

Enter marks: 110

Sample Output (Error Case)

AssertionError: Marks should be between 0 and 100.

Sample Input (Normal Case)

Valid marks entered

Sample Output (Normal Case)

Average Marks displayed

```
In [49]: try:
n = int(input("Enter number of sub marks: "))
avg = 0
total = 0

for i in range(n):
    mark = float(input(f"Enter marks of sub {i+1}: "))

    if mark < 0:
        raise ValueError("marks cannot be negative.")
    if mark > 100:
        raise ValueError("marks cannot be exceed 100.")

    total += mark
    avg = total/n

print("Total avg marks =", avg)

except ValueError as e:
    print("Error:", e)
```

Error: marks cannot be exceed 100.

10) WAP that gets password from user. Password must have at least 8 characters and one digit.

Raise WeakPasswordError if invalid, print the message "Correct" otherwise.

```
In [63]: password = input("enter your password")
try:
    if(len(password) < 8):
        raise ValueError("password length must be >8")

    count = 0
    for i in password:
        if i.isdigit():
            count += 1

    if count!=1:
        raise ValueError("password must conatain only one digit")

except ValueError as err:
    print(err)
else:
    print("correct passwords")
finally:
    print("end of the code (finally block)")
```

password must conatain only one digit
end of the code (finally block)

11) WAP to raise your custom Exception named NegativeNumberError with the error message : "Cannot calculate the square root of a negative number" :

if the given number is negative.

otherwise print the square root of the given number.

```
In [65]: import math as m
class new_Exception(Exception):
    pass

try:
    n = int(input("enter any number"))
    if(n<0):
        raise new_Exception("number not be nagetive")
    sq = m.sqrt(n)
except new_Exception as err:
    print(err)
else:
    print("square is ",sq)
finally:
    print("end of the code (finally block)")
```

number not be nagetive
end of the code (finally block)

In []: